MR B'S CTF PRACTICE

SKILLS WORKSHEETS

A one-page guide to each main challenge type — what it looks like, a worked example, and a puzzle to try. Core types first, then an Advanced section for the Hard and Expert challenges, and a Computational thinking section. Print one per pupil, or one pack per pair.

Every puzzle can be solved with the free tools on the practice site (the Toolkit → Decoder and Image inspector). An answer key is in the teacher pack.

CRYPTOGRAPHY

BASE64

HOW TO SPOT IT

A jumble of letters **and** numbers, often with one or two `=` signs on the end. Base64 is an *encoding*, not a code with a secret key — anyone can turn it straight back.

WORKED EXAMPLE

The `=` on the end is the giveaway. Send it to the Decoder and add **From Base64**:

`aGVsbG8=` → **`hello`**

Four Base64 characters stand for three normal letters, which is why the length looks odd.

YOUR TURN

Decode this flag:

```
ZmxhZ3tlYXN5X3N0YXJ0fQ==
```

🧰 **On the site:** Decoder → **From Base64**.

CRYPTOGRAPHY

BINARY (ONES & ZEROS)

HOW TO SPOT IT

Only `0`s and `1`s, usually in groups of **8**. Each group of 8 bits is one character. (Computers store every letter as a number — binary is that number written in 1s and 0s.)

WORKED EXAMPLE

Split the bits into bytes of 8, turn each into its number, then into a letter:

01100110	01101100	01100001	01100111
↓	↓	↓	↓
f	**l**	**a**	**g**

e.g. `01100110` = 102 = **f** in the ASCII table.

YOUR TURN

Decode this secret word:

```
01000110 01001100 01000001 01000111
```

🧰 **On the site:** Decoder → **From Binary**.

CRYPTOGRAPHY

HEXADECIMAL

HOW TO SPOT IT

Only the digits `0–9` and letters `a–f`, usually in pairs. Each **pair** of hex digits is one character. Hex is just another way of writing the same ASCII numbers.

WORKED EXAMPLE

Take the digits two at a time and convert each pair to a letter:

66	6c	61	67
↓	↓	↓	↓
f	**l**	**a**	**g**

e.g. `66` (hex) = 102 = **f**.

YOUR TURN

Decode this flag:

```
666c61677b6865785f74696d657d
```

🧰 **On the site:** Decoder → **From Hex**.

CRYPTOGRAPHY ROT13

HOW TO SPOT IT

Looks like real words but every letter is wrong in the **same** way. ROT13 moves each letter exactly 13 places along the alphabet. Doing it a second time brings it back.

WORKED EXAMPLE

Each letter swaps with the one 13 places along:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M

`synt` → **`flag`**

YOUR TURN

Decode this flag (tip: `synt` is `flag`):

`synt{fcva_zr}`

🧰 **On the site:** Decoder → **ROT13**.

CRYPTOGRAPHY

CAESAR SHIFT

HOW TO SPOT IT

Like ROT13, but the shift could be **any** amount from 1 to 25. When you don't know the shift, let the computer try all 25 and pick the line that reads as English.

WORKED EXAMPLE

Every letter moved forward by the same secret amount. Brute force shows all 25 shifts at once:

shift 5	… nonsense …
shift 6	… nonsense …
shift 7 ✓	**the message reads properly — flag{…}**
shift 8	… nonsense …

YOUR TURN

Find the shift that makes this readable:

```
mshn{la_ab}
```

🧰 **On the site:** Decoder → **Caesar brute force**, then read the line that makes sense.

CRYPTOGRAPHY

ATBASH (THE MIRROR)

HOW TO SPOT IT

The alphabet is flipped end to end: A becomes Z, B becomes Y, and so on. A tell-tale sign is lots of letters from the **far end** of the alphabet (z, y, x) where you'd expect common ones.

WORKED EXAMPLE

Fold the alphabet in half — top row maps to the row beneath it:

A	B	C	D	E	F	G	H	I	J	K	L	M
Z	Y	X	W	V	U	T	S	R	Q	P	O	N

`uozt` → `flag`

YOUR TURN

Un-mirror this flag:

`uozt{nriili}`

🧰 **On the site:** Decoder → **Atbash**.

CRYPTOGRAPHY

MORSE CODE

HOW TO SPOT IT

Made only of **dots** and **dashes** (and spaces between letters). Each letter is a short pattern of dots and dashes. Used on-air and in old telegraphs.

WORKED EXAMPLE

Match each group of dots and dashes to its letter using the key:

A .–	E .	N –.	R .–.	T –

`.- .-. . -. .-` → **ARENA**

YOUR TURN

Decode this word:

`-.-. -.-- -... . .-.`

🧰 **On the site:** Decoder → **Morse**.

CRYPTOGRAPHY

A1Z26 (LETTER NUMBERS)

HOW TO SPOT IT

A list of **numbers** from 1 to 26, separated by spaces or dashes. Each number is a letter's position in the alphabet: 1 = A, 2 = B … 26 = Z.

WORKED EXAMPLE

Swap each number for its letter:

1	2	3	4	5	6				
A		B		C		D		E		F

`6 12 1 7` → **FLAG**

YOUR TURN

Decode this word:

`3 9 16 8 5 18`

🧰 **On the site:** Decoder → **A1Z26**.

CRYPTOGRAPHY

XOR WITH A KEY

HOW TO SPOT IT

The data looks completely random (often shown as hex) and the challenge mentions a **key**. XOR combines each byte with the key; doing it again with the same key undoes it.

WORKED EXAMPLE

XOR compares two bits and returns 1 only when they differ:

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

Decode the hex first (**From Hex**), then **XOR with key** and type the secret word. If the key is a single unknown byte, **XOR brute force** tries all 255 for you.

YOUR TURN

On the site, decode this with **From Hex** then **XOR with key** `key` :

```
0d09180c1e010417260d101716
```

🧰 **On the site:** Decoder → **From Hex**, then **XOR with key** (or **XOR brute force**).

CRYPTOGRAPHY

RAIL FENCE (ZIG-ZAG)

HOW TO SPOT IT

All the right letters are there but in the **wrong order** — a transposition. The letters were written in a zig-zag across a number of rails, then read off one rail at a time.

WORKED EXAMPLE

Write the letters in a zig-zag, then read each row left to right to scramble; put them back on the rails to unscramble. Here is **FLAG** on 3 rails:

F			
	L		G
		A	

Read row by row: F, LG, A → scrambled. The Decoder reverses this for you.

YOUR TURN

These letters came off **3 rails** — unscramble them:

CUATRPE

🧰 **On the site:** Decoder → **Rail fence decode**, set the number of rails (try 3 first).

WEB: READ THE SOURCE

HOW TO SPOT IT

The challenge points at the **page itself** — "hidden", "look closer", "the page checks it". Your browser already has everything the page uses: text, **comments**, scripts and cookies.

WORKED EXAMPLE

School PCs block right-click & Inspect, so use the **View HTML** button on the challenge. Developers often leave notes in **comments** — the browser hides them, but they're right there:

```
<div class="card">
  <h2>Members area</h2>
  <!-- TODO remove before launch: flag{read_the_source} -->
  <p>Please log in to continue.</p>
</div>
```

Anything between `<!--` and `-->` is a comment. It never shows on screen, but it is sent to every visitor.

YOUR TURN

Find the flag hidden in the comment above and write it here:

_____

🧰 **On the site:** Open the challenge → **</> View HTML**, then read the comments and scripts.

STEGANOGRAPHY

STEGANOGRAPHY: HIDDEN IN THE PIXELS

HOW TO SPOT IT

You are given an **image** that looks blank, dark or ordinary. A message has been hidden by changing the colours very slightly — far too little for your eye to notice.

WORKED EXAMPLE

This picture looks like a flat dark rectangle. The image tools **exaggerate** tiny colour differences (or show one colour channel), and the writing jumps out:



As your eye sees it → After "exaggerate"

YOUR TURN

In your own words, which tool view would you try first on a dark, blank-looking image, and why?

🧰 **On the site:** Open the image in the **Image inspector** → try **Exaggerate tiny differences** or a single colour channel (Red / Green / Blue only).



ADVANCED TECHNIQUES

The skills behind the Hard and Expert challenges: multi-step decoding, the Vigenère cipher, guessing a XOR key from a crib, and brute-force thinking. Try the core worksheets first.

CRYPTOGRAPHY

VIGENÈRE CIPHER

ADVANCED

HOW TO SPOT IT

Looks like a Caesar shift — real-looking letters all moved along — but **Caesar brute force never fully reads**. That is because the shift *changes* along the message: a repeating secret **keyword** sets a different shift for each letter.

WORKED EXAMPLE

The keyword is written over and over beneath the message; each letter of the key says how far to shift. So you must know (or guess) the key:

H	E	L	L	O
K	E	Y	K	E

The decoder does the shifting — your job is to spot that it is Vigenère and find the key (often hinted in the challenge).

YOUR TURN

Decode this word — the key is `arena`:

```
vzggorp
```

🧰 **On the site:** Decoder → **Vigenère decode**, then type the key in the key box.

ANY CATEGORY

PEELING THE LAYERS

ADVANCED

HOW TO SPOT IT

The data has been encoded **several times over**. The trick is to read the *shape* after each step and work from the outside in — one decode at a time.

WORKED EXAMPLE

Learn the fingerprints, then undo them one by one:

`=` on the end	→ Base64
only 0-9 and a-f	→ Hex
letters look mirrored (z, y, x)	→ Atbash
words almost read, all shifted the same	→ ROT13 / Caesar
right letters, wrong order	→ Rail fence (transposition)

After **every** decode, look again and ask "what is this now?"

YOUR TURN

Peel this one (hint: `=` first, then 0-9 a-f):

```
NjY2YzYxNjc3YjcwNjU2NTZjNWY2ZDY1N2Q=
```

🧰 **On the site:** Add one operation at a time in the Decoder; re-read the output before choosing the next.

CRYPTOGRAPHY

XOR: GUESS THE KEY

ADVANCED

HOW TO SPOT IT

Random-looking bytes (usually hex) XOR-ed with a **repeating key you aren't told**. One-byte brute force won't help — but you have a secret weapon: every flag starts `flag{`.

WORKED EXAMPLE

XOR is its own inverse, so if `cipher = text ⊕ key` then `key = cipher ⊕ text`. XOR the first cipher bytes against the letters you already know to read the key straight off:

`cipher bytes ⊕ f l a g {` → **`k e y k e …`**

The key repeats, so a few known letters reveal all of it. This is a "known-plaintext" attack.

YOUR TURN

From Hex first. The flag starts `flag{` — use that to find the 3-letter key, then XOR:

```
151c18140b011c0226100210110d
```

🧰 **On the site:** Decoder → **From Hex**, work out the key from the `flag{` crib, then **XOR with key**.

PROGRAMMING / REVERSING

WHEN STUCK, TRY EVERYTHING

ADVANCED

HOW TO SPOT IT

Some challenges use a formula, PIN or number that **can't be run backwards**. When you can't reverse it, make the computer **try every possibility** — that is a brute-force.

WORKED EXAMPLE

Computers are fast, so exhausting the options is often trivial:

Caesar	only 25 shifts — try them all
1-byte XOR	only 255 keys — try them all
4-digit PIN	only 10,000 codes — try them all

```
// JavaScript scratchpad
for (let n = 0; n < 10000; n++) {
  if ((n*7 + 13) % 10000 === 8651) console.log(n);
}
```

Caesar and XOR brute force are built into the Decoder; for number puzzles use the JS scratchpad.

YOUR TURN

Run the loop above (or reason it out): which `n` from 0–9999 makes `(n×7 + 13) mod 10000 = 8651`?

🧰 **On the site:** Decoder → **Caesar brute force** / **XOR brute force**; or the Toolkit **JS scratchpad** for a loop.



COMPUTATIONAL THINKING

Logic puzzles with nothing to decode — the thinking skills behind every challenge: spotting patterns, tracing algorithms, Boolean and bitwise logic, and reasoning from clues.

LOGIC & THINKING

SEQUENCES & PATTERNS

THINKING

HOW TO SPOT IT

A list of numbers that follow a **hidden rule**. The skill is **pattern recognition**: look at what changes from one number to the next, then test your rule on the numbers you can already see.

WORKED EXAMPLE

Write the difference (or the operation) underneath each gap, then continue it:

2, 4, 8, 16 …	each one ×2 → next is 32
1, 3, 6, 10 …	gaps +2, +3, +4 → next gap +5 → 15

If one rule doesn't fit every step, it's the wrong rule — try another.

YOUR TURN

Find the next number (hint: look at the gaps):

```
1, 2, 4, 7, 11, ?
```

🧰 **On the site:** No tools needed — work it on paper, then type `flag{your_number}` on the Logic challenges.

LOGIC & THINKING

TRACING AN ALGORITHM

THINKING

HOW TO SPOT IT

A few lines of pseudocode with a **loop**. Don't try to do it in your head — keep a **trace table** and write down each variable's value after every step.

WORKED EXAMPLE

Example: `x = 3`, then `repeat 3 times: x = x + 2`. Trace it:

start	loop 1	loop 2	loop 3
3	5	7	9

The answer is the **last** value in your table: 9.

YOUR TURN

Trace this and give the final `total`:

```
total = 0
for n from 1 to 4:
    total = total + n × n
```

🧰 **On the site:** Trace on paper; check with the Toolkit JS scratchpad if you like.

LOGIC & THINKING

BOOLEAN & BITWISE LOGIC

THINKING

HOW TO SPOT IT

Words like **AND, OR, NOT**, or binary numbers combined bit by bit. Learn the three rules and apply them one column at a time.

WORKED EXAMPLE

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

AND = both 1; OR = at least one 1; XOR = exactly one 1. For binary numbers, apply the rule to each column.

YOUR TURN

Work out this bitwise XOR (1 where the bits differ):

```
  1101
XOR 0110
= ????
```

🧰 **On the site:** Do it by hand; the Logic challenges want the binary answer, e.g. `flag{1010}`.

LOGIC & THINKING

LOGIC & DEDUCTION

THINKING

HOW TO SPOT IT

No maths — just clues. Everything you need is in the question. Start from the clue that **fixes** one fact, write down what *must* be true, and cross off everything it rules out.

WORKED EXAMPLE

Three pupils like one subject each (Maths, Art, PE). "Ben likes Maths; Amy is not PE."

`Ben = Maths`	`(given — so nobody else has Maths)`
`Amy = Art`	`(not PE, and Maths is taken)`
`Cara = PE`	**`(the only option left)`**

YOUR TURN

Tom, Sam and Pat each like one colour: red, blue or green. *Sam likes green. Tom does not like red.* **Who likes red?**

🧰 **On the site:** Work it out on paper; answer the Logic challenges with the name or word, e.g. `flag{cara}`.